Task 1 — Composables Practice Reflection & Planning

Student Name:Krishna Lagamappa umarani
File Name: krishnaLagamappaUmarani_Task1_ComposablesPractice.pdf

1. Codelab Problem Reviews

Problem 1: Compose Article Layout

- **Summary:** Build a UI that displays a tutorial article featuring a full-width header image at the top, followed by a large title, a short introductory paragraph, and a longer main body paragraph.
- **Planned Approach:** I would use a Column to stack all elements vertically. The first element would be an Image composable with fillMaxWidth(). The next three elements would be Text composables. I would apply a padding(16dp) modifier to the text elements so they don't touch the screen edges.
- **Actual Results & Reflection:** I built it mostly as planned. However, I initially found it tricky to make the image span the exact width without distorting the aspect ratio. I realized I needed to use contentScale = ContentScale.FillWidth in the Image parameters. Next time, I will review the documentation on image scaling *before* coding.

Problem 2: Task Manager "Completed" Screen

- **Summary:** Build a screen showing that a user has completed all tasks. It requires a prominent checkmark image in the center, a bold "All tasks completed" text below it, and a smaller "Nice work!" text below that.
- **Planned Approach:** I would use a single Column wrapped around an Image and two Text composables. To center everything on the screen, I'd apply fillMaxSize() to the Column, and set verticalArrangement = Arrangement.Center and horizontalAlignment = Alignment.CenterHorizontally.
- **Actual Results & Reflection:** The plan worked perfectly. I found that Jetpack Compose makes centering elements much easier compared to old XML constraints. The trickiest part was remembering the difference between Arrangement (for the primary axis) and Alignment (for the cross axis).

3. Planning Checklist Template

When solving a new composable UI problem, I will use this checklist:

1. **Gather Spec:** Identify all required text, images, icons, and fonts.
2. **Sketch/Deconstruct Layout:** Draw boxes around UI components to visualize where I need Row, Column, or Box layouts.

3. **Pick Base Composables:** Map the sketch elements to Compose UI components (Text, Image, Icon, Button).
4. **Define Hierarchy & Alignment:** Decide how child elements align within their parent containers (e.g., Arrangement.SpaceBetween).
5. **Assign Modifiers:** Plan size limits (fillMaxWidth), spacing (padding), shapes (clip), and colors.
6. **Test on Previews:** Set up Android Studio @Preview annotations for Light/Dark mode and Phone/Tablet sizes.

4. Short Reflection

Planning and reflection drastically reduce the amount of "trial-and-error" coding. By visualizing the UI as a tree of nested rows and columns before touching the keyboard, I write cleaner, more structured Compose code. Reflecting on mistakes (like mismanaging image scaling) builds my mental model so I don't repeat the same errors in future projects.

Task 2 — Design & Document a Business Card Layout (Pre-Code)

Student Name: Krishna lagamappa umarani
File Name: krishnaLagamappaUmarani_Task2_BusinessCardDesign.pdf

1. Concept

Concept Choice: "Android Developer Portfolio Card"

2. Concept Details

- **Title & Subtitle:** Krishna l u (KT) | Junior Android Developer
- **Short Bio:** Passionate about building clean, accessible, and user-centric mobile experiences using Kotlin and Jetpack Compose.
- **Contact Fields:**
  - Email: guru@email.com
  - GitHub: github.com/alexdoedev
  - Phone: +91 9876543210
- **Image:** A circular professional headshot.

3. Wireframe / Sketch Plan (Textual Representation)

- **Layout Structure:**
  - Root: Surface (for background color)
  - Main Container: Column (centered using fillMaxSize and Arrangement.SpaceEvenly)

        - **Top Section (Profile):** Column (centered horizontally)
        - Image (Headshot) -> Modifier: size(120dp), clip(CircleShape)
        - Text (Name) -> Modifier: padding(top = 16dp)
        - Text (Subtitle) -> Modifier: padding(top = 8dp)
        - **Bottom Section (Contact Info):** Column (centered)
        - Row 1: Icon(Phone) + Spacer + Text
        - Row 2: Icon(Email) + Spacer + Text
        - Row 3: Icon(GitHub) + Spacer + Text
        - *(Modifiers for Rows: padding(8dp), fillMaxWidth(0.8f) to keep them aligned nicely in the center)*
- **Tablet/Landscape Variant Sketch Note:** Instead of one tall Column, the Root will be a Row. The left half of the screen will hold the Top Section (Profile + Bio), and the right half of the screen will hold the Bottom Section (Contact Info), horizontally centered within their respective halves using weight(1f).

4. Resource & Style Plan

- **Images:**
    - profile_headshot.jpg (placed in res/drawable-nodpi or multiple density buckets).
    - Default Material Icons for Phone, Email, and Share/Web.
- **Text Style Tokens:**
    - Name: 32sp, FontWeight.Light ○ Subtitle: 16sp, FontWeight.Bold ○ Contact Info: 14sp, FontWeight.Normal □ **Colour Palette:**
    - Primary text: Dark Charcoal (#202020) ○ Subtitle text: Android Green (#3DDC84) ○ Background: Very Light Mint (#F0F4F1) □ **Accessibility Plan:**
    - contentDescription = "Profile Picture of Alex Doe" for the headshot. ○ contentDescription = "Phone Number" for the phone icon, etc. ○ Ensure each contact Row has a minimum height modifier of 48.dp so they meet touch-target accessibility standards if I make them clickable later.

5. Short Reflection

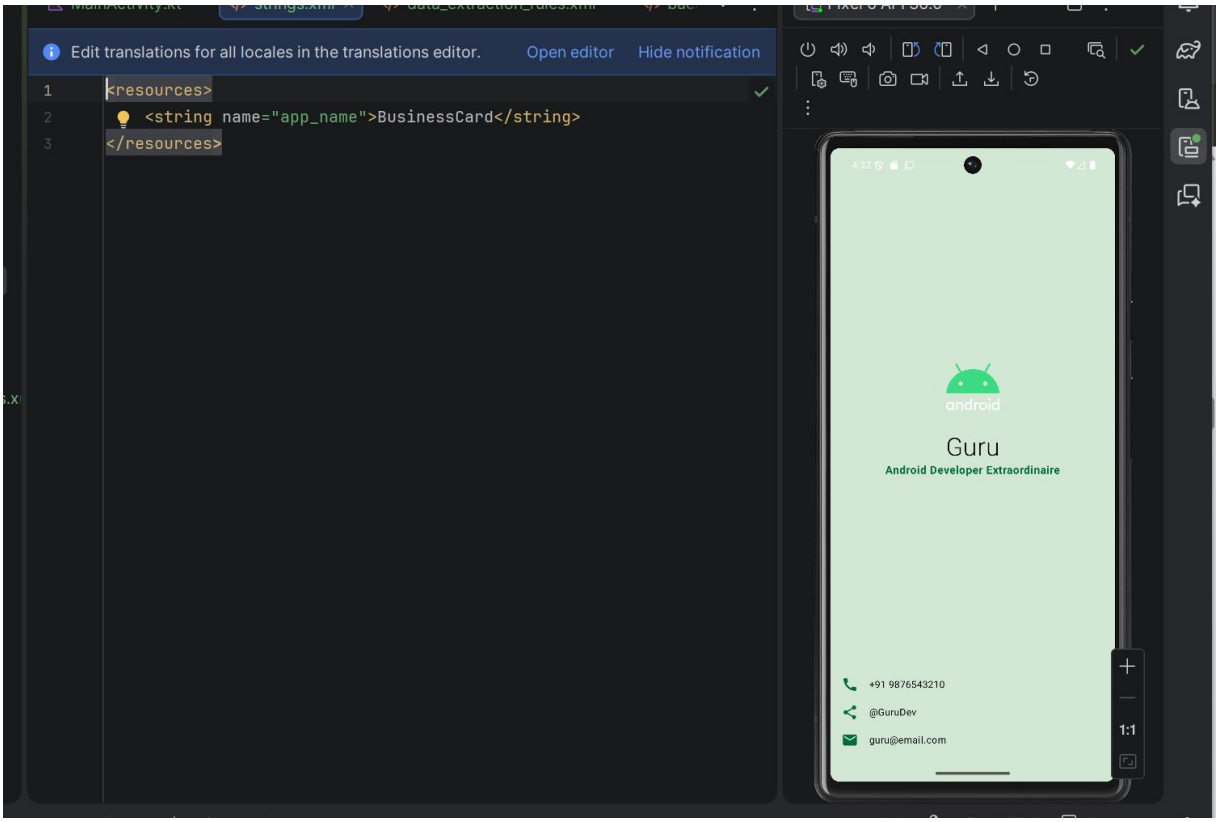
I chose this layout design because it clearly separates the "who I am" from the "how to reach me" into visual blocks. I am heavily relying on Column and Row to structure the data, and padding to give the elements room to breathe. I will test this before deploying by creating multiple @Preview functions in Android Studio (one for standard phone, one for landscape, and one with a large font size).

Task 3 — Layout Review & Improvement Plan

Student Name: Krishna lagamappa umarani
File Name: krishnaLagamappaUmarani_Task2_BusinessCardDesign.pdf

1. Screenshots

(Note: If submitting the actual document, you would paste your two screenshots here: 1. Phone Portrait, 2. Tablet Landscape. For this text template, we will assume they are attached).



1 *Phone Portrait,*

4:32
android
Guru
Android Developer Extraordinaire
+91 9876543210
@GuruDev
guru@email.com
1:1
1:1
LF
UTF-8
4 spaces

2. Review Using Criteria

- **Visual alignment & spacing:** The main profile image and name are centered perfectly. However, the contact rows at the bottom look slightly staggered because the text lengths vary. They should ideally be aligned to a specific starting guideline rather than just centered as a block.
- **Readability:** Contrast is excellent (Dark Charcoal on Light Mint background). The name text is legible. The contact info text (14sp) feels slightly too small on a dense mobile screen.
- **Resource usage:** I used vector icons for the contact details, which scale perfectly without pixelation. My profile photo was originally a 3MB JPG; I need to compress this and convert it to WebP to save space.
- **Responsiveness:** In portrait, it looks great. In Tablet Landscape, the elements are stretched too far apart, leaving awkward empty whitespace at the top and bottom of the screen.
- **Accessibility & usability:** Icons have content descriptions, which is great. However, padding on the contact rows is only 8dp, meaning the clickable height is about 30dp. This fails the 48dp minimum touch target guideline.

3. Improvement Plan

Issue Found	Proposed Fix	Priority
Contact rows fail touch-target size rules	Add Modifier.heightIn(min = 48.dp) or increase padding to contact Rows.	**High**
Profile image file is too large (3MB)	Convert profile_headshot.jpg to WebP format and compress to < 200kb.	**High**
Awkward whitespace Landscape mode	Implement a layout switch. If width > height, use a Row to in place profile on the left and contact info on the right.	**Medium**
Contact details text slightly too small	Increase font size from 14sp to 16sp for better legibility on small screens.	**Medium**
Contact rows are staggered	Put the Rows inside a Column that has fillMaxWidth(0.6f) to left-align the icons while keeping the block centered on screen.	**Low**

4. Reflection

I learned that a layout that looks visually "fine" on a single preview might actually fail basic accessibility checks (like touch target sizing) or perform poorly on larger form factors. In future Compose projects, I will bake the 48dp touch-target rule and landscape preview testing into my initial coding phase, rather than fixing them at the end.